Guion de trabajo tutorizado

Introducción a la gestión de datos masivos

Gestión y visualización de datos masivos

Profesor Dr. Carlos D. Barranco González
Máster Big Data & Business Analytics (MBDA)





ÁLVARO RAMOS BERNABÉ

E1. (120 minutos – 3.3 puntos) ¿Cuáles son los problemas que enfrentan las bases de datos tradicionales?

Jacobs (2009) parte de un experimento: un censo ficticio de 6.750 millones de personas (100 GB) se procesa con un programa secuencial sencillo en 15 minutos, pero PostgreSQL no logra cargarlo y un GROUP BY sobre 1.000 millones de filas tarda más de 24 horas. Las patologías que identifica son:

1. **Es más fácil meter los datos que sacarlos.** *Causa:* los SGBD están diseñados para el procesamiento transaccional: insertar, actualizar y recuperar pocos registros. *Efecto:* almacenar no es el problema; lo es **analizar**. El optimizador eligió ordenar mil millones de filas en lugar de agregarlas con una tabla hash.
2. **Inflación del almacenamiento.** *Causa:* la representación interna del SGBD ocupa 3-4 veces lo necesario. *Efecto:* la carga se abortó a las 6 horas con el disco casi lleno.
3. **Patrones de acceso no secuenciales.** *Causa:* el modelo relacional ignora el orden de las filas, aunque los datos masivos crecen por observaciones repetidas en el tiempo y el espacio. *Efecto:* cuando los datos no caben en memoria las lecturas se vuelven aleatorias, y el acceso aleatorio a disco es unas 150.000 veces más lento que el secuencial (el aleatorio a RAM es incluso más lento que el secuencial a disco). Por eso las *joins* entre tablas grandes no ordenadas por la clave resultan carísimas y, paradójicamente, conviene **desnormalizar**.
4. **Límites duros del software.** *Causa:* herramientas que cargan todo en memoria (R) o con topes arbitrarios (65.536 filas en Excel, índices de 32 bits). *Efecto:* se quedan fuera del problema aunque el hardware pudiera resolverlo, lo que contradice la regla «cero, uno o infinito».
5. **Costes de la distribución.** *Causa:* la solución natural, repartir datos y cálculo entre servidores baratos, exige que el procesamiento sea local a cada nodo. *Efecto:* el reparto desigual de la carga (el nodo más lento marca el tiempo), el conflicto entre particionar por tiempo o por entidad y la menor fiabilidad (diez nodos, diez veces más averías), que obliga a replicar.

Conclusiones. Las patologías del big data son sobre todo **de análisis y de acceso a los datos**, no de CPU ni de almacenamiento. Jacobs propone una metadefinición: big data es aquel cuyo tamaño obliga a abandonar los métodos habituales del momento. En mi opinión, el artículo anticipa la motivación de NoSQL, los motores columnares y MapReduce: ordenar y particionar según la consulta, leer secuencialmente y llevar el cálculo a los datos.

E2. (120 minutos – 3.3 puntos) ¿Qué es ACID? ¿Qué es BaSE? ¿Qué diferencias hay entre los dos? ¿Sirven para lo mismo? ¿Dan las mismas garantías?

¿Qué es ACID?

Es el modelo de control de transacciones de los SGBD relacionales (McCreary y Kelly, 2014, cap. 2). Garantiza **Atomicidad** (la transacción se ejecuta completa o no se ejecuta), **Consistencia** (de un estado válido a otro, respetando las reglas de integridad), **Aislamiento** (las transacciones concurrentes no ven resultados intermedios, para lo que se usan bloqueos) y **Durabilidad** (lo confirmado sobrevive a caídas). El ejemplo clásico es

una transferencia bancaria: el cargo y el abono se producen juntos o ninguno. Lo implementan Oracle, PostgreSQL o MySQL (InnoDB).

¿Qué es BaSE?

Es el enfoque de muchos sistemas NoSQL distribuidos: **Basically Available** (el sistema siempre responde, aunque algún nodo falle), **Soft state** (el estado puede cambiar sin nuevas escrituras mientras se propagan las réplicas) y **Eventual consistency** (si cesan las actualizaciones, todas las réplicas acaban convergiendo al mismo valor). Lo aplican, por ejemplo, Amazon Dynamo, Cassandra o Riak.

¿Qué diferencias hay entre los dos?

ACID es **pesimista**: asegura la consistencia antes de responder, bloqueando recursos, y por ello escala mal horizontalmente. BaSE es **optimista**: acepta lecturas momentáneamente desactualizadas a cambio de disponibilidad, baja latencia y escalado en muchos nodos. El teorema CAP explica el compromiso: ante una partición de red, un sistema distribuido debe elegir entre consistencia (ACID) y disponibilidad (BaSE).

¿Sirven para lo mismo?

No. ACID es el adecuado cuando un dato incorrecto tiene un coste inaceptable: banca, stock, facturación. BaSE conviene cuando es peor no responder que responder con un dato ligeramente antiguo: carritos de compra, catálogos, contadores, redes sociales o *logs*. Muchas arquitecturas combinan ambos.

¿Dan las mismas garantías?

No. ACID garantiza **consistencia inmediata y fuerte** en cada transacción; BaSE garantiza **disponibilidad** y solo **consistencia eventual**, y delega en la aplicación la resolución de conflictos entre versiones. Hoy existen posiciones intermedias (consistencia ajustable por operación, o NewSQL con ACID distribuido), pero la elección sigue dependiendo del coste de negocio de un dato inconsistente frente al de un servicio caído.

E3. (60 minutos – 1.7 puntos) ¿Qué motor NoSQL es el más apropiado para cada aplicación?

Para determinar qué base de datos es la más apropiada, no podemos basarnos simplemente en la popularidad. Según el material docente, la elección depende de una reflexión sobre la naturaleza del proyecto, considerando factores críticos como la complejidad de los datos, el volumen de operaciones (lectura/escritura) y la necesidad de escalabilidad.

A continuación, presento la tabla comparativa que cruza los modelos estudiados con los factores determinantes, incluyendo el modelo relacional como punto de referencia.

🟢 **Total:** El modelo es excelente o nativo para este factor.

🟡 **Parcial:** El modelo lo soporta con limitaciones o mediante complejidad adicional.

🔴 **Nulo/Bajo:** El modelo no está diseñado para esto o presenta un rendimiento pobre.

Factor / Modelo	Escalabilidad Horizontal (Volumen masivo)	Flexibilidad de Esquema (Datos no estructurados)	Complejidad de Relaciones (Datos muy conectados)	Consultas Ad-Hoc / Complejas	Caso de Uso Típico (Ejemplos)
Relacional (RDBMS)	🔴 (Escala verticalmente)	🔴 (Esquema rígido)	🟢 (Joins potentes)	🟢 (SQL estándar)	Financiero, Facturación
Clave-Valor	🟢 (Muy alta)	🟢 (Totalmente agnóstico)	🔴 (Sin relaciones)	🔴 (Solo por clave)	Sesiones de usuario, Carritos de compra
Documental	🟢 (Alta)	🟢 (JSON/XML flexible)	🟡 (Sin Joins, anidamiento)	🟡 (Consultas por atributos)	Catálogos de productos, CMS
Familias de Columnas	🟢 (Extrema)	🟢 (Columnas dinámicas)	🔴 (Sin Joins)	🟡 (Optimizado para lectura por fila)	Analítica masiva, Logs de actividad
Grafos	🟡 (Más difícil de particionar)	🟢 (Atributos en nodos/aristas)	🟢 (Nativo y performante)	🟢 (Navegacionales)	Motores de recomendación, Redes sociales

Relacional: Sigue siendo el rey para la integridad referencial y transacciones complejas (ACID), pero falla en escalabilidad horizontal y flexibilidad ante datos cambiantes.

Clave-Valor: Es el modelo más simple y veloz, ideal cuando el acceso es siempre por una clave primaria conocida (ej. ID_Sesión), pero inútil si necesitas buscar por el contenido del valor.

Documental: Ofrece un equilibrio "dulce". Permite cierta estructura compleja (documentos anidados) sin la rigidez de las tablas, siendo ideal para objetos como fichas de producto.

Columnar: Su fortaleza reside en la escritura masiva y la analítica sobre grandes volúmenes de datos dispersos, donde añadir columnas es trivial.

Grafos: Es la única opción viable cuando el valor de los datos reside en sus conexiones (quién conoce a quién), resolviendo consultas que harían colapsar a un RDBMS por exceso de *joins*

E4. (60 minutos – 1.7 puntos) ¿Qué es NewSQL? ¿En qué se diferencia de NoSQL? ¿Cuáles son los tipos de NewSQL que podemos encontrar en el mercado? ¿Es mejor NewSQL que NoSQL? ¿Queda NoSQL obsoleto? ¿Hay algo más allá de

NewSQL en lo que se refiere a bases de datos? ¿Cuáles son las novedades y diferencias con respecto a NewSQL?

¿Qué es NewSQL?

NewSQL es una categoría de sistemas de gestión de bases de datos (DBMS) que busca alcanzar el "santo grial" del almacenamiento: ofrecer la escalabilidad horizontal y flexibilidad características de los sistemas NoSQL, pero reteniendo las garantías ACID y el soporte para consultas SQL de las bases de datos tradicionales.

Es como intentar tener un coche deportivo (rápido y ágil como NoSQL) pero con la seguridad y los controles de un tanque (robusto y fiable como RDBMS).

¿En qué se diferencia de NoSQL?

La diferencia fundamental radica en la **consistencia**. Mientras que NoSQL suele adoptar el modelo BASE (sacrificando consistencia inmediata por disponibilidad y particionado), NewSQL se adhiere estrictamente a ACID y al lenguaje SQL estándar. NewSQL no renuncia a la integridad transaccional para escalar.

¿Cuáles son los tipos de NewSQL que podemos encontrar en el mercado?

El ecosistema es variado. Encontramos soluciones que son nuevos motores construidos desde cero y otros que actúan como capas de mejora sobre sistemas existentes. Entre los ejemplos destacados en el mercado se encuentran VoltDB, NuoDB, Clustrix, ScaleDB y Xeround.

¿Es mejor NewSQL que NoSQL? ¿Queda NoSQL obsoleto?

No es necesariamente "mejor", sino diferente, y NoSQL no queda obsoleto. La industria se mueve hacia la Persistencia Políglota, donde se combinan tecnologías en un mismo proyecto según la necesidad. Se usaría NewSQL para los datos financieros críticos (facturación) y NoSQL para los logs de actividad de usuarios, aprovechando lo mejor de cada uno.

¿Hay algo más allá de NewSQL en lo que se refiere a bases de datos?

Sí, la evolución natural que trasciende al NewSQL inicial es el SQL Distribuido (Distributed SQL).

¿Cuáles son las novedades y diferencias con respecto a NewSQL?

El SQL Distribuido se define como una base de datos lógica única desplegada a través de múltiples nodos físicos, incluso en diferentes centros de datos. A diferencia de las primeras iteraciones de NewSQL (que a veces eran parches de *sharding*), el SQL Distribuido ofrece resiliencia a prueba de balas y una escala elástica nativa, permitiendo que la base de datos sobreviva a fallos completos de zonas geográficas sin perder la integridad.

Bibliografía

[1] Jacobs, A. (2009). The pathologies of big data. *Communications of the ACM*, 52(8), 36-44. https://doi.org/10.1145/1536616.1536632

[2] McCreary, D. y Kelly, A. (2014). *Making sense of NoSQL: A guide for managers and the rest of us*. Manning. Capítulos 1 y 2.

[3] Brewer, E. (2012). CAP twelve years later: How the "rules" have changed. *Computer*, 45(2), 23-29.